In [1]:
```
import numpy as np
def gauss_p(sa,sb):
    ni, nj = sa.shape
    x = np.zeros(ni)
    if ni != nj:
        print('not rectangular matrix')
        return x
    else:
        n = ni
    if n != sb.shape[0]:
        print('no vector b')
        return x
    a = np.zeros([n,n])
    a[:,:] = sa[:]
    b = np.zeros(n)
    b[:] = sb[:]

    eps = np.finfo(np.float).eps * np.amax(np.abs(np.diag(a)))
    for j in range (0, n-1):
        if abs(a[j,j]) < eps*1e5:
            if np.amax(np.abs(a[j+1:n,j])) < eps:
                print('zero pivot')
                return x
            loc = np.argmax(np.abs(a[j+1:n,j])) + 1
            if j == 0:
                a[j:j+1+loc:loc,j:] = a[j+loc::-loc,j:]
                b[j:j+1+loc:loc] = b[j+loc::-loc]
            else:
                a[j:j+1+loc:loc,j:] = a[j+loc:j-1:-loc,j:]
                b[j:j+1+loc:loc] = b[j+loc:j-1:-loc]
        for i in range (j+1, n):
            p = a[i,j] / a[j,j]
            a[i,j+1:n] -= p * a[j,j+1:n]
            b[i] -= p * b[j]
    for i in range (n-1, -1, -1):
        x[i] = (b[i] - np.sum(a[i,i+1:n]*x[i+1:n])) / a[i,i]
    return x
```

In [2]:
```
n = 100
rt = np.random.rand(n,n)
ra = rt.astype(np.float64)
sa = ra + 0.
rb = np.random.rand(n)
sb = rb + 0.
x = gauss_p(ra,rb)
y = np.linalg.solve(sa,sb)
print(np.sum(x-y)/(n*n))
print(np.sum(np.dot(sa,x)-sb))
print(np.sum(np.dot(sa,y)-sb))
```

```
6.222713316850203e-18
4.784506124622112e-12
-1.1002310174035301e-13
```

In [3]:
```
n = 100
rt = np.random.rand(n,n)
ra = rt.astype(np.float64)
sa = ra + 0.
rb = np.random.rand(n)
sb = rb + 0.
%timeit x = gauss_p(ra,rb)
%timeit y = np.linalg.solve(sa,sb)
```

```
22.6 ms ± 2.02 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)
341 µs ± 47.1 µs per loop (mean ± std. dev. of 7 runs, 1000 loops each)
```

In [4]:
```
%load_ext Cython
```

In [5]:
```
%%cython
import numpy as np
cimport cython
DTYPE = np.double
@cython.boundscheck(False)
@cython.wraparound(False)
cpdef gauss_c(double [:,:] sa, double[:] sb):
    cdef Py_ssize_t ni, nj, n, j, i, loc, k
    cpdef double eps, p

    ni = sa.shape[0]
    nj = sa.shape[1]
    x_np = np.zeros(ni, dtype=DTYPE)
    cdef double [:] x = x_np

    if ni != nj:
        print('not rectangular matrix')
        return x_np
    else:
        n = ni
    if n != sb.shape[0]:
        print('no vector b')
        return x_np

    a_np = np.zeros([n,n], dtype=DTYPE)
    cpdef double [:,:] a = a_np
    a[:,:] = sa[:,:]
    b_np = np.zeros(n, dtype=DTYPE)
    cpdef double [:] b = b_np
    b[:] = sb[:]

    eps = np.finfo(np.float).eps * np.amax(np.abs(np.diag(a)))
    for j in range (0, n-1):
        if abs(a[j,j]) < eps*1e5:
            if np.amax(np.abs(a[j+1:n,j])) < eps:
                print('zero pivot')
                return x_np
            loc = np.argmax(np.abs(a[j+1:n,j])) + 1
#            print('pivoting lines=',j,j+loc)
            if j == 0:
                a[j:j+1+loc:loc,j:] = a[j+loc::-loc,j:]
                b[j:j+1+loc:loc] = b[j+loc::-loc]
            else:
                a[j:j+1+loc:loc,j:] = a[j+loc:j-1:-loc,j:]
                b[j:j+1+loc:loc] = b[j+loc:j-1:-loc]
        for i in range (j+1, n):
            p = a[i,j] / a[j,j]
            for k in range (j+1, n):
                a[i,k] = a[i,k] - p * a[j,k]
            b[i] = b[i] - p * b[j]
    for i in range (n-1, -1, -1):
        x[i] = (b[i] - np.sum(a[i,i+1:n]*x_np[i+1:n])) / a[i,i]
    return x_np
print('done compiling')
```

```
done compiling
```

In [6]:
```
n = 100
ra = np.random.rand(n,n)
rb = np.random.rand(n)
x = gauss_c(ra,rb)
y = np.linalg.solve(ra,rb)
print(np.sum(x-y)/(n*n))
print(np.sum(np.dot(ra,x)-rb))
print(np.sum(np.dot(ra,y)-rb))
```

```
5.639655409339638e-17
1.2372325386422744e-12
1.417754802446325e-13
```

In [7]:
```
n = 100
ra = np.random.rand(n,n)
rb = np.random.rand(n)
%timeit x = gauss_c(ra,rb)
%timeit y = np.linalg.solve(ra,rb)
```

```
1.1 ms ± 7.61 µs per loop (mean ± std. dev. of 7 runs, 1000 loops each)
357 µs ± 50.1 µs per loop (mean ± std. dev. of 7 runs, 1000 loops each)
```

In [8]:
```
n = 1000
ra = np.random.rand(n,n)
rb = np.random.rand(n)
%timeit x = gauss_c(ra,rb)
%timeit y = np.linalg.solve(ra,rb)
```

```
384 ms ± 12.6 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
23.8 ms ± 411 µs per loop (mean ± std. dev. of 7 runs, 10 loops each)
```

In [ ]:
```
```